



Institución
Universitaria
Reacreditada en Alta Calidad

ESTRUCTURA DE DATOS Y »»» LABORATORIO «««

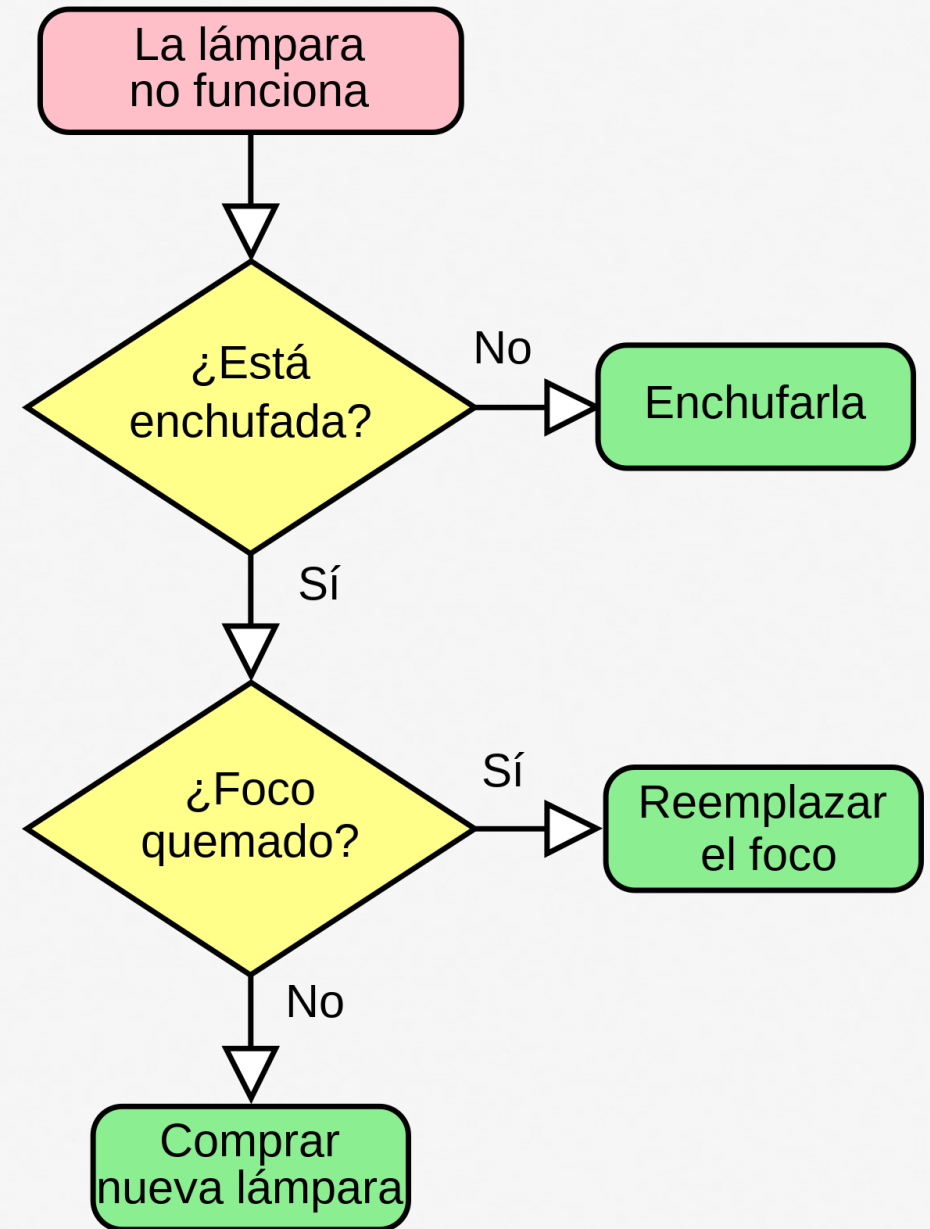
580506004-8

William Giraldo Escobar
Facultad de Ingenierías
Departamento de Sistemas de Información



QUÉ ES UN ALGORITMO?

- Un algoritmo es una secuencia de pasos computacionales que transforman la entrada en la salida.



PYTHON

- Para construir estructuras de datos necesitamos darle instrucciones detalladas a una computadora. La mejor forma de realizar esta comunicación es utilizando un *lenguaje de alto nivel* como Python.
- Los *lenguajes de alto nivel* son los lenguajes de programación que más se aproximan al lenguaje humano.

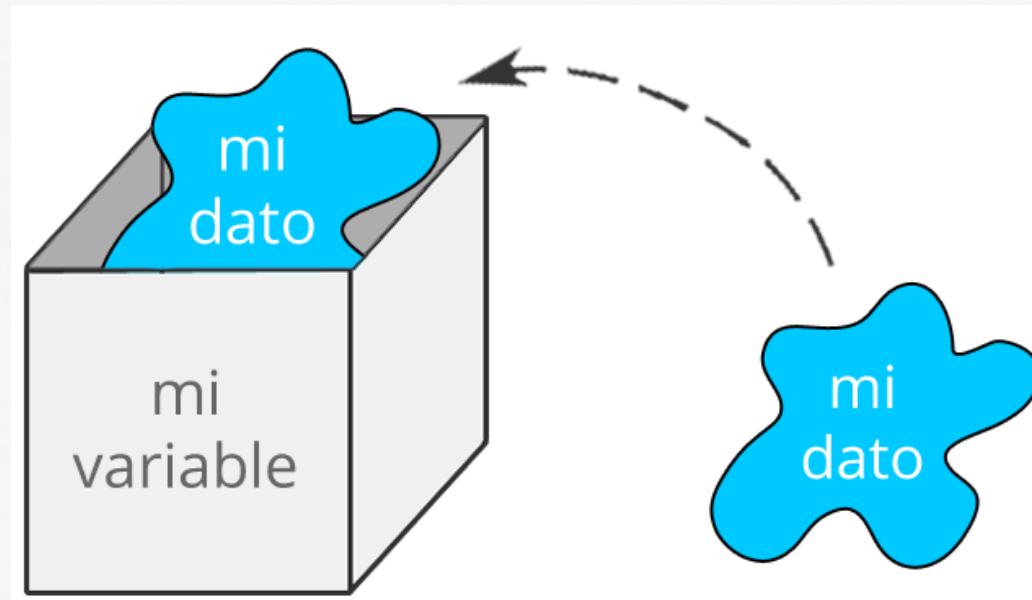
ESTRUCTURA DE DATOS CON PYTHON

Como prerrequisito para el curso de estructura de datos usted debe dominar los conceptos de:

- Variables y expresiones
- Condicionales
- Ciclos
- Funciones
- Vectores y matrices

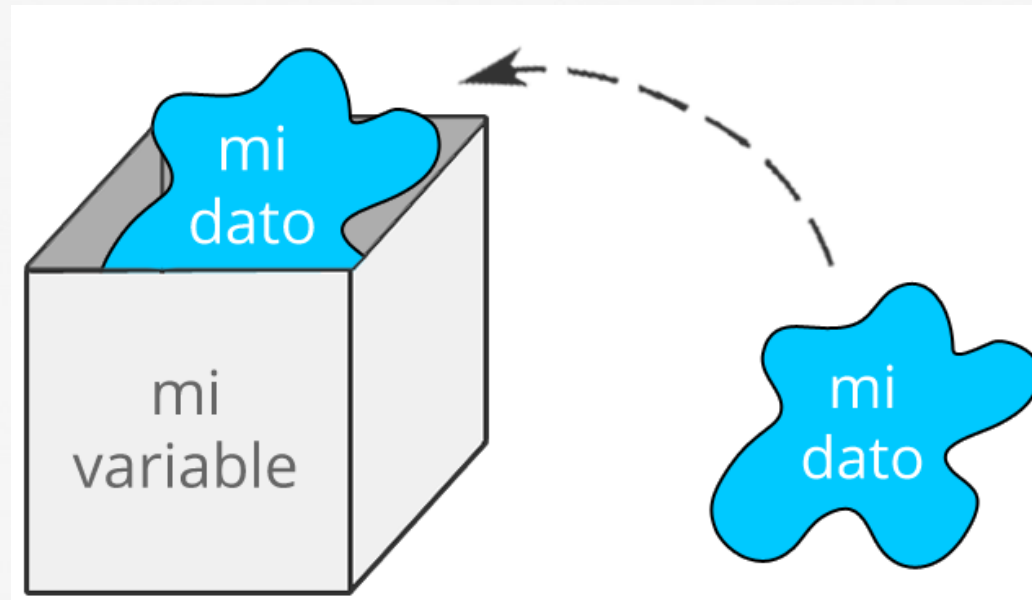
VARIABLES

- Una variable es un nombre simbólico que se utiliza para representar un dato almacenado en un espacio de memoria del ordenador.



VARIABLES

- Los datos almacenados en la variable se pueden manipular y cambiar durante la ejecución del programa.



Identificadores

Son los nombres que usamos para nombrar a las variables, deben seguir las siguientes reglas:

- No empezar por un numero (3happy)
- No contener caracteres especiales, excepto el guion bajo '_' que sí está permitido
- No puede ser una de las 33 palabras reservadas por python

PALABRAS RESERVADAS

Reserved Words								
False	as	continue	else	from	in	not	return	yield
None	assert	def	except	global	is	or	try	
True	break	del	finally	if	lambda	pass	while	
and	class	elif	for	import	nonlocal	raise	with	



CLASES

Los datos almacenados en las variables pueden pertenecer a distintos tipos de clase.

La clase de cada variable depende del formato del dato que se almacene.

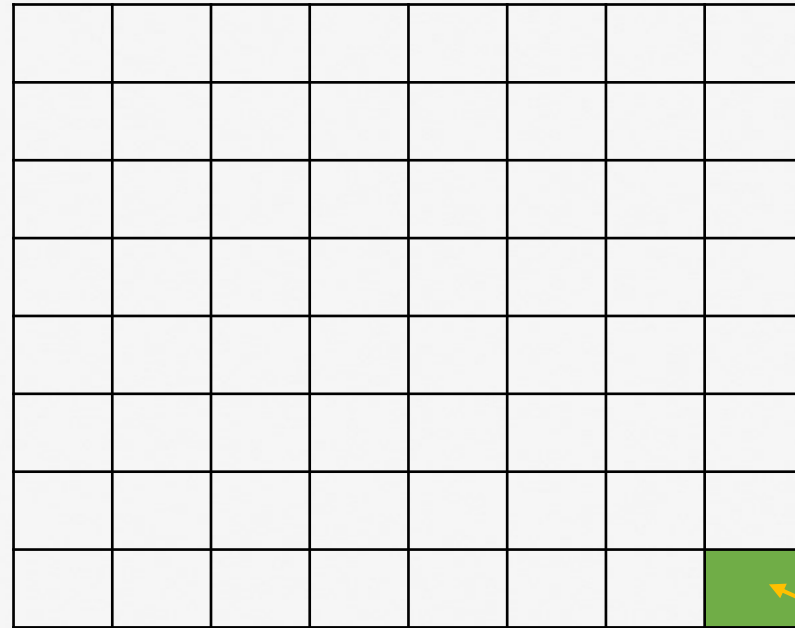
CLASES DE VARIABLES EN PYTHON

Class	Description	Immutable?
bool	Boolean value	✓
int	integer (arbitrary magnitude)	✓
float	floating-point number	✓
list	mutable sequence of objects	
tuple	immutable sequence of objects	✓
str	character string	✓
set	unordered set of distinct objects	
frozenset	immutable form of set class	✓
dict	associative mapping (aka dictionary)	

CLASES DE VARIABLES EN PYTHON

- **int:** 4 bytes (32 bits) , 8 bytes (64 bits)
- **float:** 8 bytes (64 bits)
- **Bool:** TRUE or FALSE (1 bit)
- **str:** su tamaño depende del contenido

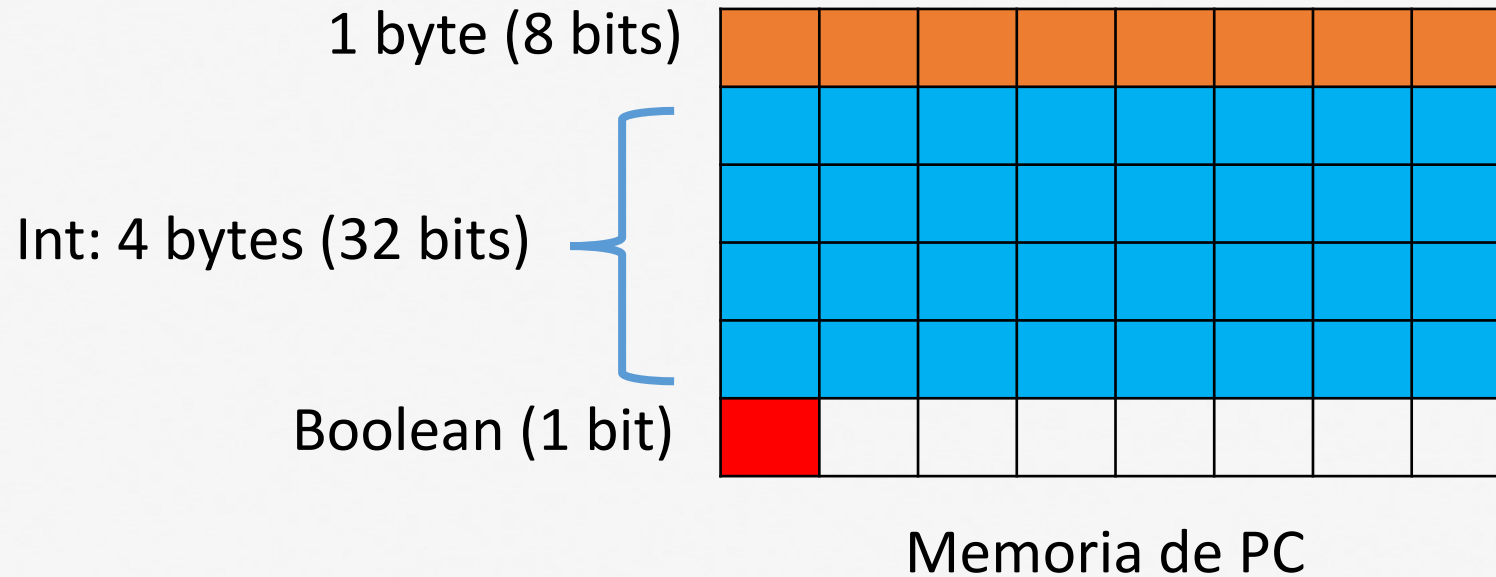
CLASES DE VARIABLES EN PYTHON



Memoria de PC

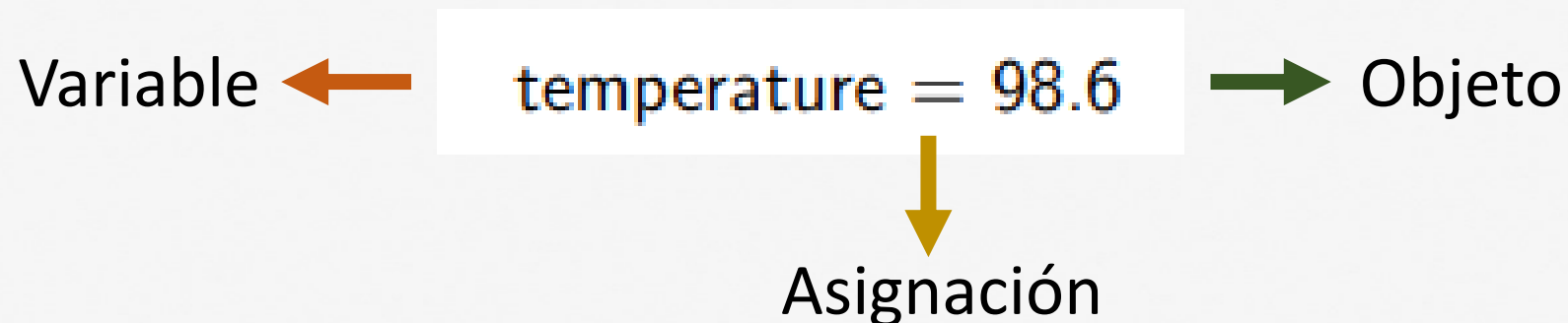
Cada posición
de memoria es
1 bit
(0 o 1)

CLASES DE VARIABLES EN PYTHON



VARIABLES, OBJETOS Y ASIGNACIÓN

- El objeto es el dato que se almacena en una variable
- La asignación es el proceso en el que almacenamos un dato dentro de una variable

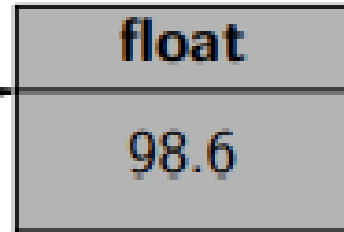


CLASE DE UNA VARIABLE

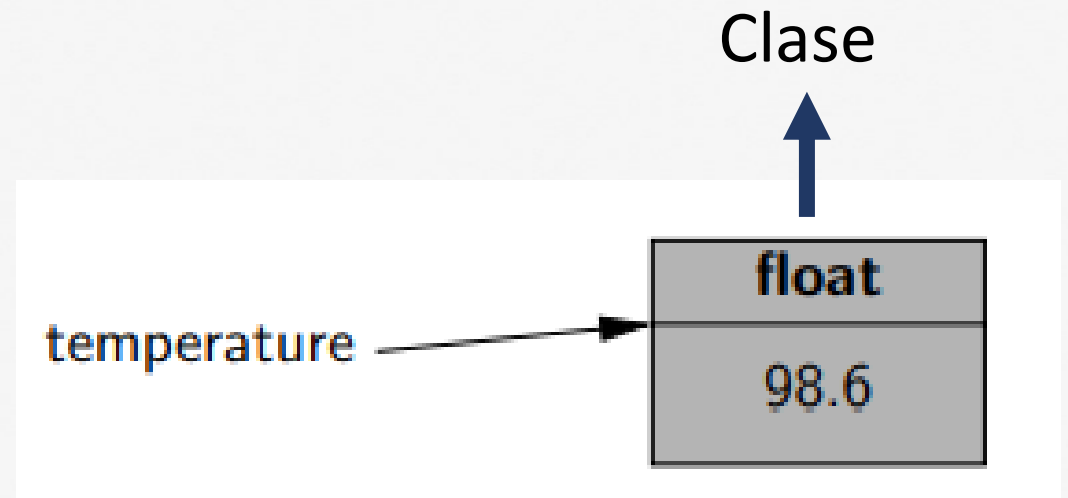
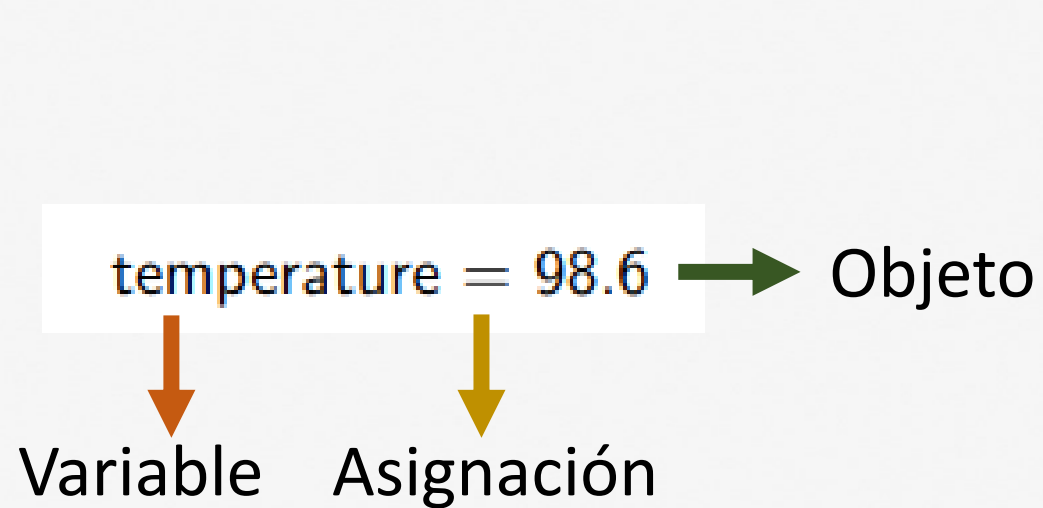
En el ejemplo, la variable 'temperature' es una instancia de la clase float

```
temperature = 98.6
```

temperature



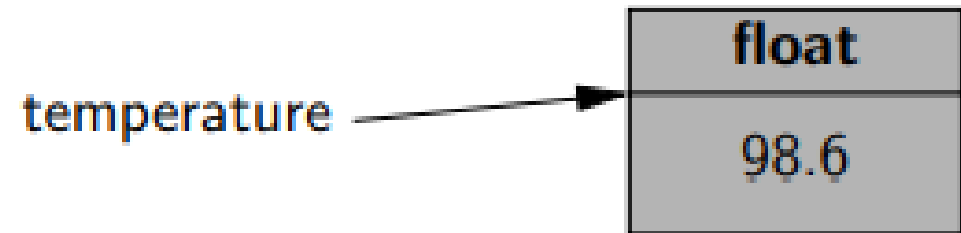
VARIABLE, OBJETO, ASIGNACIÓN, CLASE



NOTA

- Tener en cuenta que en Python, para la creación de una variable no se necesita declarar el tipo de dato (int, float, str) como en otros lenguajes de programación.
- Sin embargo, **NO PODEMOS OLVIDAR** que el objeto que pertenece a esa variable **SÍ** pertenece a una clase (int, float, str).

```
temperature = 98.6
```



FUNCIÓN TYPE()

- La usamos para comprobar a qué *clase* pertenece una *variable*

```
# Tipos básicos
numero = 42
cadena = "Hola, mundo"
lista = [1, 2, 3]
tupla = (4, 5, 6)
diccionario = {"clave": "valor"}

type(numero)    # <class 'int'>
type(cadena)     # <class 'str'>
type(lista)      # <class 'list'>
type(tupla)      # <class 'tuple'>
type(diccionario) # <class 'dict'>
```

EXPRESIONES Y OPERADORES

- Las expresiones son combinaciones de valores, variables, operadores y llamadas a funciones que se evalúan para producir un resultado.
- Las expresiones pueden ser simples, como una operación aritmética, o más complejas, involucrando múltiples operaciones y valores.
- Los operadores, por otro lado, son símbolos o palabras clave que se utilizan para realizar operaciones en las expresiones.

OPERADORES LOGICOS

not	unary negation
and	conditional and
or	conditional or

x	y	x or y
False	False	False
False	True	True
True	False	True
True	True	True

x	y	x and y
False	False	False
False	True	False
True	False	False
True	True	True

x	not x
False	True
True	False

Or: al menos una afirmación debe ser TRUE para obtener un resultado tipo TRUE

and: ambas afirmaciones deben ser TRUE para obtener un resultado tipo TRUE



OPERADORES LOGICOS

and

a = True

b = False

resultado = a and b

OPERADORES LOGICOS

and

a = True

b = False

resultado = a and b *# Resultado será False, ya que b es False*



OPERADORES LOGICOS

or

```
x = True
```

```
y = False
```

```
resultado = x or y
```

OPERADORES LOGICOS

or

```
x = True
```

```
y = False
```

```
resultado = x or y # Resultado será True, ya que x es True
```



OPERADORES LOGICOS

not

z = True

resultado = not z



OPERADORES LOGICOS

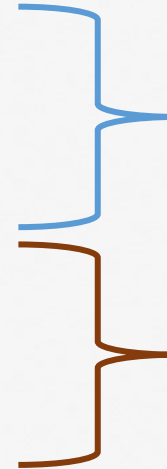
not

z = True

resultado = not z *# Resultado será False, ya que z es True*

OPERADORES DE IGUALDAD

is	same identity
is not	different identity
==	equivalent
!=	not equivalent



Compara la misma posición de memoria

Compara el mismo contenido



OPERADORES DE IGUALDAD

==

```
a = 5
```

```
b = 10
```

```
resultado = a == b
```

OPERADORES DE IGUALDAD

==

```
a = 5
```

```
b = 10
```

```
resultado = a == b # Resultado será False, ya que a no es igual a b
```



OPERADORES DE IGUALDAD

!=

```
x = "Hola"  
y = "Mundo"  
resultado = x != y
```

OPERADORES DE IGUALDAD

!=

```
x = "Hola"
```

```
y = "Mundo"
```

```
resultado = x != y # Resultado será True, ya que x es diferente de y
```

OPERADORES DE IGUALDAD

is

```
lista1 = [1, 2, 3]
```

```
lista2 = lista1
```

```
resultado = lista1 is lista2
```

```
print(resultado) # Esto imprimirá True, ya que lista1 y lista2 son la misma instancia en memoria
```

OPERADORES DE IGUALDAD

Is not

```
cadena1 = "Hola"
```

```
cadena2 = "Hola"
```

```
resultado = cadena1 is not cadena2
```

```
print(resultado) # Esto imprimirá True, ya que cadena1 y cadena2  
no son la misma instancia en memoria
```


OPERADORES DE COMPARACIÓN

<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to

OPERADORES DE COMPARACIÓN

```
numero1 = 25
```

```
numero2 = 15
```

```
resultado = numero1 > numero2  
print(resultado)
```

OPERADORES DE COMPARACIÓN

```
edad1 = 30
```

```
edad2 = 30
```

```
resultado = edad1 >= edad2
```

```
print(resultado)
```

OPERADORES DE COMPARACIÓN

```
cantidad1 = 100
```

```
cantidad2 = 150
```

```
resultado = cantidad1 <= cantidad2
```

```
print(resultado)
```

OPERADORES ARITMÉTICOS

+	addition
—	subtraction
*	multiplication
/	true division
//	integer division
%	the modulo operator

OPERADORES ARITMÉTICOS

```
a = 5  
b = 3
```

```
suma = a + b  
print(suma)
```

```
a = 5  
b = 3  
print(a + b)
```

```
print(5+3)
```

OPERADORES ARITMÉTICOS

Cuál es la diferencia?

```
dividendo = 15  
divisor = 3
```

```
division = dividendo / divisor  
print(division)
```

```
dividendo = 15  
divisor = 4
```

```
division_entera = dividendo // divisor  
print(division_entera)
```

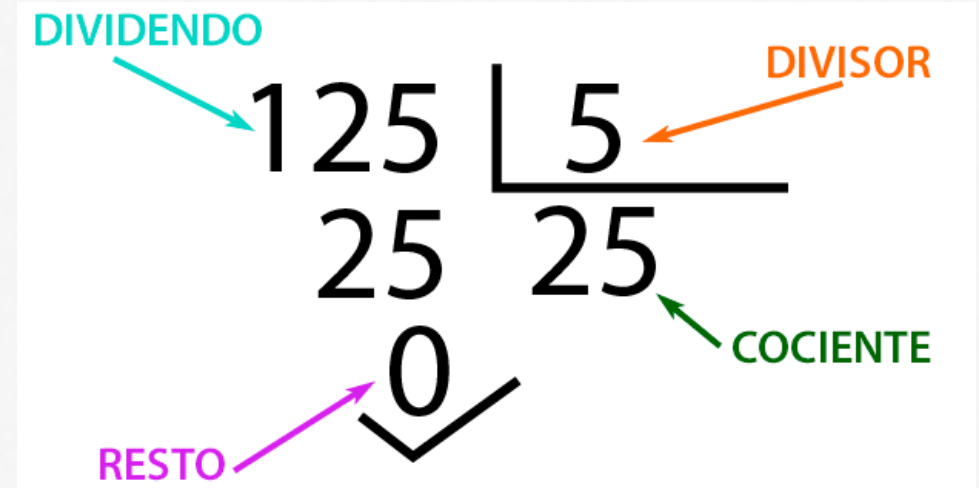
OPERADORES ARITMÉTICOS

```
numero = 20
```

```
divisor = 7
```

```
modulo = numero % divisor
```

```
print(modulo)
```



OPERADORES ARITMÉTICOS

```
base = 2  
exponente = 3  
  
potencia = base ** exponente  
print(potencia)
```

OPERADORES DE ASIGNACIÓN

Operator	Example	Equivalent Expression (m=15)	Result
=	y = <u>a+b</u>	y = 10 + 20	30
+=	m +=10	m = m+10	25
-=	m -=10	m = m-10	5
*=	m *=10	m = m*10	150
/=	m /=10	m = m/10	1.5
%=	m %=10	m = m%10	5
=	m **=2	m = m2 or $m = m^2$	225
//=	m //=10	m = m//10	1

OPERADORES DE ASIGNACIÓN

Declarar m=15

Operator	Example
=	y = <u>a+b</u>
+=	m +=10
-=	m -=10
*=	m *=10
/=	m /=10
%=	m %=10
=	m=2
//=	m//=10

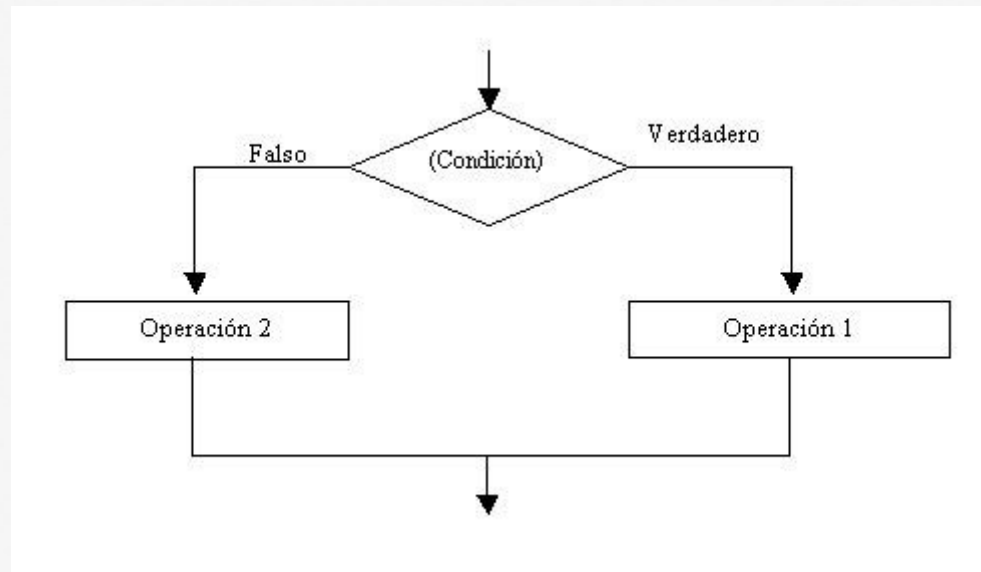
Result
30
25
5
150
1.5
5
225
1

NOTA

- Las operaciones tienen un orden de prioridad:
- Paréntesis: ()
- Exponenciación: **
- Multiplicación (*), División (/), División entera (//), Módulo (%)
- Suma (+), Resta (-)

CONDICIONALES

- Los condicionales permiten que un programa tome diferentes caminos de ejecución dependiendo de ciertas circunstancias.



CONDICIONAL IF EN PYTHON

Seudocódigo

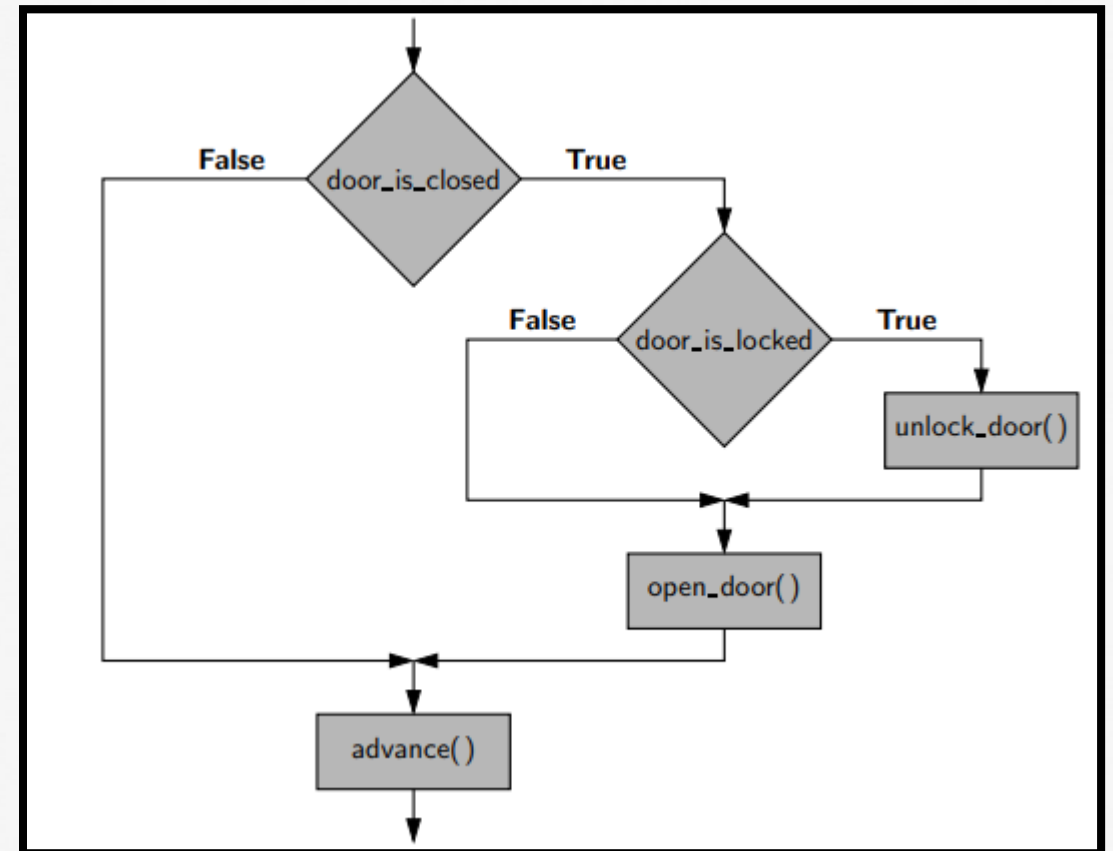
```
Si primera condición entonces
    operaciones
Sino
    si segunda condición entonces
        operaciones
    Sino
        Operación por defecto
Fin Si
```

Python

```
if first_condition:
    first_body
elif second_condition:
    second_body
elif third_condition:
    third_body
else:
    fourth_body
```

CONDICIONALES: UNA CORRECTA IDENTACION

```
if door_is_closed:  
    if door_is_locked:  
        unlock_door()  
    open_door()  
    advance()
```



CONDICIONALES

- Las reglas condicionales se expresan a través de comparaciones o expresiones booleanas

Ejemplo de comparaciones:

- $x \geq 5$: x es mayor igual a cinco?
- $z == 0$: z es igual a cero?
- $y \neq 1$: y es diferente de uno?

CONDICIONALES

Expresiones booleanas

- Las expresiones booleanas permiten combinar una o más comparaciones
- Emplean los operadores NOT (!), OR (|), y AND (&)

Ejemplo de expresiones booleanas:

- $!(x \geq 5)$: x no es mayor igual a cinco?
- $z == 0 \mid y \neq 1$: z es igual a cero o y es diferente de uno?
- $y \neq 1 \ \& \ x \geq 5$: y es diferente de uno y x es mayor igual a cinco

EJEMPLO CONDICIONALES - *NOTEBOOK*

Seudocódigo

```
Si x>=y & x>=z entonces
    Retornar x
sino
    si y>=x & y>=z entonces
        Retornar y
Sino
    Retornar z
Fin si
```

Python

```
if x>=y & x>=z:
    return x
elif y>=x & y>=z:
    return y
else:
    return z
```

EJEMPLO CONDICIONALES

```
temperatura = 25
lluvia = False

if temperatura > 20 and not lluvia:
    print("Es un buen día para salir")
else:
    print("Mejor quedarse en casa")
```

EJEMPLO CONDICIONALES

```
Edad=  
if edad < 18:  
    print("Eres menor de edad")  
elif edad >= 18 and edad < 65:  
    print("Eres adulto")  
else:  
    print("Eres un adulto mayor")
```

EJEMPLO CONDICIONALES ANIDADOS

```
usuario =  
contrasena =  
  
if usuario == "admin":  
    if contrasena == "secreto":  
        print("¡Bienvenido, administrador!")  
    else:  
        print("Contraseña incorrecta para el usuario admin")  
else:  
    print("Usuario desconocido")
```



CICLOS

- Son estructuras que permiten ejecutar un bloque de código repetidamente según una condición específica.
- Los ciclos son útiles para automatizar tareas que deben realizarse múltiples veces sin tener que repetir el mismo código una y otra vez.

CICLO WHILE

- Lo usamos cuando desconocemos las veces que necesitamos repetir una expresión.
- Se controla por una condición de parada

```
while condition:  
    body
```

CICLO WHILE

Seudocódigo

Inicializar contador

Mientras *condición de parada* **Hacer**

Operaciones

Incrementar contador

Fin Mientras

Python

```
contador = 0
```

```
while contador < 5:
```

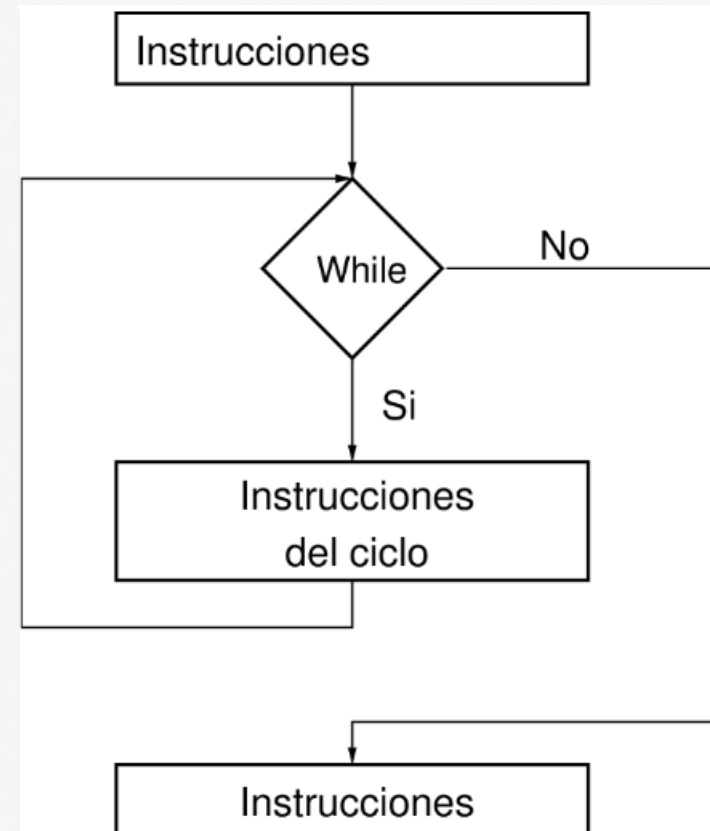
```
    print("Número:", contador)
```

```
    contador += 1
```


CICLO WHILE: UNA CORRECTA IDENTIFICACIÓN

```
valor = 10
```

```
while valor >= 2:  
    print("Valor:", valor)  
    valor -= 1  
print("fin del programa")
```



CICLO WHILE: CONDICIONES DE PARADA

Contador y Límite:

- Una de las formas más comunes de utilizar un bucle while es con un contador y un límite. El bucle se ejecuta mientras el contador es menor que un valor límite.

```
contador = 0
limite = 5
while contador < limite:
    # Código a ejecutar en
    cada iteración
    contador += 1
```

CICLO WHILE: CONDICIONES DE PARADA

Entrada del Usuario:

- Puedes usar la entrada del usuario para determinar cuándo detener el ciclo. Por ejemplo, puedes solicitar al usuario que ingrese una respuesta y luego evaluar esa respuesta en la condición del ciclo.

```
respuesta = input("¿Quieres continuar? (s/n): ")  
while respuesta == 's':  
    # Código a ejecutar en cada iteración  
    respuesta = input("¿Quieres continuar? (s/n): ")
```

CICLO WHILE: CONDICIONES DE PARADA

Condiciones Combinadas:

- Puedes combinar múltiples condiciones usando operadores lógicos como and y or. Esto te permite tener condiciones más complejas para controlar el ciclo.

```
contador = 0
limite = 5
respuesta = 's'
while contador < limite and respuesta == 's':
    # Código a ejecutar en cada iteración
    contador += 1
    respuesta = input("¿Quieres continuar? (s/n): ")
```

EJEMPLO WHILE - *NOTEBOOK*

Seudocódigo

```
Contador=0  
Mientras contador <5 Hacer  
    imprimir contador  
    Incrementar contador  
Fin Mientras
```

Python

```
contador = 0  
while contador < 5:  
    print("Número:", contador)  
    contador += 1
```

EJEMPLO WHILE - *NOTEBOOK*

```
activo = True

while activo:
    respuesta = input("¿Quieres continuar? (sí/no): ")

    if respuesta.lower() == "no":
        activo = False
        print("Saliendo del programa.")
    else:
        print("Continuando...")
```

CICLO FOR

- Lo usamos cuando conocemos las veces que necesitamos repetir una expresión.
- Se debe crear un objeto iterable para utilizarlo (vector, lista..)

```
for element in iterable:  
    body
```

CICLO FOR

Seudocódigo

```
lista = [10, 20, 30, 40, 50] // Lista de elementos

para cada elemento en lista hacer
    imprimir("Elemento: " + elemento)
fin para
```

Python

```
lista = [10, 20, 30, 40, 50] # Lista de elementos

for elemento in lista:
    print("Elemento:", elemento)
print("Fin del programa")
```


CICLO FOR – OBJETOS ITERABLES

listas

```
frutas = ["manzana", "banana", "naranja"]  
for fruta in frutas:  
    print(fruta)
```

CICLO FOR – OBJETOS ITERABLES

rangos

```
for numero in range(1, 6): # Itera desde 1  
    hasta 5 (no incluye 6)  
    print(numero)
```

CICLO FOR – OBJETOS ITERABLES

cadenas

```
mensaje = "Hola, mundo!"  
for caracter in mensaje:  
    print(caracter)
```

CICLO FOR – OBJETOS ITERABLES

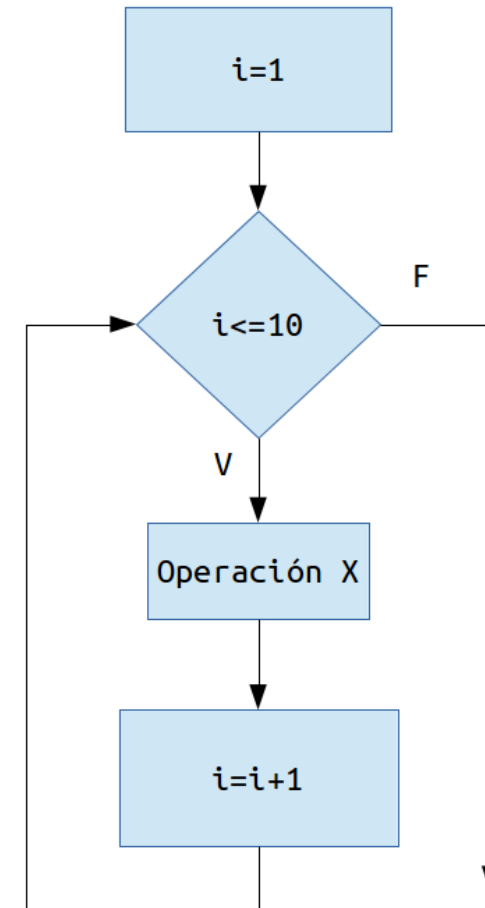
'enumerate' puedo iterar usando dos variables en un mismo vector,
Con una variable recorro los datos y con la otra las posiciones de esos datos

```
nombres = ["Juan", "María", "Carlos"]  
for indice, nombre in enumerate(nombres):  
    print(f"Índice: {indice}, Nombre: {nombre}")
```

CICLO FOR: UNA CORRECTA IDENTIFICACIÓN

```
numeros = [2, 7, 14, 21, 36, 45]
```

```
for numero in numeros:  
    if numero % 2 == 0:  
        print(numero, "es par")  
    else:  
        print(numero, "es impar")
```



EJEMPLO WHILE - *NOTEBOOK*

Seudocódigo

```
lista = [10, 20, 30, 40, 50] // Lista de elementos
```

```
para cada elemento en lista hacer
```

```
    imprimir elemento
```

```
fin para
```

Python

```
lista = [10, 20, 30, 40, 50] # Lista de elementos
```

```
for elemento in lista:
```

```
    print("Elemento:", elemento)
```

```
print("Fin del programa")
```

LA FUNCION RANGE

- Se utiliza para generar una secuencia de números. Esta secuencia es comúnmente utilizada en ciclos for para iterar sobre un rango específico de valores. La función range() acepta uno, dos o tres argumentos y devuelve un objeto iterable que produce una secuencia de números enteros.

range(inicio, fin, paso)

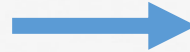
LA FUNCION RANGE

- inicio: El valor inicial de la secuencia (por defecto es 0 si no se proporciona).
- fin: El valor final de la secuencia (no se incluye en la secuencia generada).
- paso: El incremento entre los números en la secuencia (por defecto es 1 si no se proporciona).

range(inicio, fin, paso)

LA FUNCION RANGE

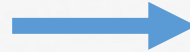
```
for numero in range(5):  
    print(numero)
```



Salida
[0,1,2,3,4]

LA FUNCION RANGE

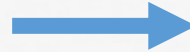
```
for numero in range(1, 11):  
    print(numero)
```



Salida
[1,2,3,4,5,6,7,8,9,10]

LA FUNCION RANGE

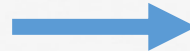
```
for numero in range(0, 11, 2):  
    print(numero)
```



Salida
[0,2,4,6,8,10]

LA FUNCION RANGE

```
for numero in range(10, 0, -1):  
    print(numero)
```



Salida
[10,9,8,7,6,5,4,3,2,1]

ROMPER UN CICLO - BREAK

- Se utiliza en ciclos for y while, para salir del bucle antes de que se complete su ciclo normal.
- Se utiliza para detener la ejecución del ciclo en un punto específico cuando se cumple una cierta condición.

```
found = False
for item in data:
    if item == target:
        found = True
        break
```

ROMPER UN CICLO - BREAK

Seudocódigo

```
para cada elemento en lista hacer  
    si elemento es igual a 5 entonces  
        imprimir("Se encontró el número 5")  
        romper // Interrumpir el bucle  
    fin si  
fin para  
  
imprimir("Fin del programa")
```

Python

```
numeros = [2, 4, 6, 8, 10]  
  
for numero in numeros:  
    if numero == 6:  
        print("Se encontró el número 6")  
        break # Salir del bucle  
print("Fin del programa")
```




Institución
Universitaria
Reacreditada en Alta Calidad

¡MUCHAS GRACIAS!

A decorative graphic element consisting of several horizontal lines in various colors (yellow, orange, red, purple, blue, green) and a stylized white swoosh or arc on the right side, positioned below the '¡MUCHAS GRACIAS!' text.

..... Hacia una era de
Universidad y *Humanidad*